LaunchLens
Development Challenges and Solutions

Evidence-grounded SDE interview preparation report

Based on repository code and Git history. Claims described only in the brief without repository evidence are clearly marked Not verified.

Executive summary

LaunchLens is a React/TypeScript and Express/TypeScript campaign analytics MVP. Users create projects and campaign redirect links; a public GET /r/:publicSlug records a ClickEvent with a browser-scoped visitor UUID, User-Agent-derived device/browser, referrer and timestamp, then sends a 302 redirect. Prisma persists User -> Project -> Campaign -> ClickEvent data to PostgreSQL. Analytics are deterministic; an AI layer summarizes those trusted facts through Gemini, Groq, and Mistral fallback.

The most valuable interview challenges are cross-origin production authentication, deployment configuration, resilient AI output handling, browser metadata collection, and honest analytics-data quality boundaries. Git commits specifically record CORS, production login, Render, Vercel, and Axios fixes; current source confirms the final implementations.

Challenge	Area	Severity	Final solution / evidence
Production CORS origin mismatch	Deployment	High	CORS uses FRONTEND_URL + credentials; Git history records repeated CORS/frontend URL fixes.
Cross-site session cookie	Authentication	High	Production cookie is Secure + SameSite=None; commit 8a38084 says fixed login production.
Local vs deployed API endpoint	Frontend/deployment	High	Axios distinguishes localhost VITE_API_URL from production /api proxy; commit 6de0843.
AI provider failure/invalid JSON	AI	High	AIProvider router tries Gemini -> Groq -> Mistral and validates JSON with Zod.
Browser Unknown	Tracking	Medium	Redirect parses User-Agent with UAParser and persists browser.
Country Unknown	Analytics data quality	Medium	No IP-geolocation code exists; country stays Unknown and is no longer in AI context.
Schema-only future features	Scope/design	Medium	Conversion and UTM-like fields retained in schema but not misrepresented as implemented.

Contents

1. Evidence and complete issue timeline

Evidence standard. Verified means current source or Git history directly supports the implementation or fix. **Not verified** means the request mentions an exact runtime incident, but no error log, commit diff, or source record proves its occurrence; it is useful only as a discussion prompt, not as a story to claim as fact.

Sequence (Git)	Verified change / inferred problem
Aug 3: redirect engine and UUID	Commits 6c23664 and 6a60a10: schema changed; redirect engine and UUID unique-visitor tracking added.
Aug 3: analytics	Commits ec5d0b3, 3f7e27c, 092a00e: click counts, backend aggregation, and frontend charts implemented.
Aug 9: AI	Commit fc8f9c7: AI integration and fallback added; current router validates JSON and falls through providers.
Aug 9: deployment	Commits a4ef95b, 637203d, f3a1b8a, 2504e9f: frontend URL/CORS revisions.
Aug 9: production login	Commit 8a38084 changes cookie SameSite Lax to None in production and clears with matching options.
Aug 9: Render/Vercel/Axios	Commits 65519dd, 7afa710, 6de0843 record Render, Vercel rewrite, and Axios baseURL changes.

CORS configuration evolution

Status: Verified
Stage: Production integration

```
EXPECTED
Deployed Vercel client should send credentialed API requests to Render.

ACTUAL
Git history contains multiple frontend URL/CORS fixes, including 2504e9f "fixed cors issue".

INVESTIGATION -> ROOT CAUSE -> FIX -> VERIFICATION
Current app.ts has cors({ origin: process.env.FRONTEND_URL, credentials: true }); prior commit allowed an explicit
localhost + Vercel list. This proves configuration changed during deployment, but the exact OPTIONS header values
requested in the brief are not stored in repository history.

SOLUTION
Set FRONTEND_URL to the deployed frontend origin, redeploy the backend, and use exact origin matching with
credentials. Confirm in browser Network panel that Origin and Access-Control-Allow-Origin match.
```

Lesson learned: CORS is a browser policy: TCP/API availability is not enough. Treat Origin, response headers, and cookie attributes as a single cross-origin contract.

Production login cookie handling

Status: Verified
Stage: Production authentication

```
EXPECTED
Login should persist the auth cookie so GET /api/auth/me can restore the session.

ACTUAL
Commit 8a38084 is titled "fixed login issue in production" and changes cookie settings.

INVESTIGATION -> ROOT CAUSE -> FIX -> VERIFICATION
Earlier production configuration used sameSite: lax. Cross-site frontend/backend deployment needs Secure +
SameSite=None for browser credential cookies.

SOLUTION
Login and logout now derive isProd from NODE_ENV or HTTPS FRONTEND_URL; they set/clear the HTTP-only token with
secure: isProd and sameSite: none in production. Axios has withCredentials: true.
```

Lesson learned: Cookie attributes must match the browser topology, and logout must clear with compatible attributes.

2. Production CORS and authentication

CORS story: verified versus requested exact incident

Verified: local server prints localhost:PORT; frontend Axios sends credentials; backend CORS takes FRONTEND_URL; Git commits show the deployed Vercel domain was introduced and CORS was repeatedly fixed. **Not verified:** the exact Network capture in the brief - OPTIONS 204 with Access-Control-Allow-Origin: http://localhost:5173 while Origin was https://launch-lens-beta.vercel.app - is not present in repository, logs, or Git diff. Do not quote that header pair as personally observed unless you have an external screenshot/log.

If that capture occurred, its diagnosis is straightforward: the preflight OPTIONS can correctly return 204 yet still fail CORS because the response permits a different origin. With credentials:true/withCredentials:true, Access-Control-Allow-Origin must be a specific matching origin; wildcard * is invalid for credentialed browser access. The browser then blocks the actual response from JavaScript even if the server is reachable.

```
Browser Network debugging playbook
1. Select OPTIONS /api/auth/login.
2. Read Request Headers: Origin.
3. Read response Access-Control-Allow-Origin and Access-Control-Allow-Credentials.
4. Compare exact scheme + host + port.
5. Check POST request and Set-Cookie.
6. Change FRONTEND_URL / cookie options, redeploy Render, retest from Vercel.
```

Authentication flow and 401 investigation

```
POST /api/auth/login -> Zod -> user lookup -> bcrypt.compare
-> jwt.sign({ userId }, JWT_SECRET, 7d) -> Set-Cookie token (HTTP-only)
-> browser sends cookie with Axios withCredentials -> GET /api/auth/me
-> authenticate reads req.cookies.token -> jwt.verify -> req.user.userId -> user query
```

401 occurs when token is missing, invalid, or expired; middleware responds "Authentication required" or "Invalid or expired token". The verified production fix specifically addresses cookie transport, not password/JWT logic. To debug, distinguish: no Set-Cookie; cookie blocked by SameSite/Secure/domain; cookie not sent because withCredentials/CORS; or cookie sent but JWT verification fails.

Interview-ready CORS answer

30 seconds: "During deployment, the browser and API moved to different origins. I treated it as a browser policy issue, not a backend availability issue. I aligned the backend CORS allowed origin with FRONTEND_URL, enabled credentials, used Axios withCredentials, and adjusted production auth cookies to Secure and SameSite=None. Then I verified the actual request and response headers in the Network tab."

1 minute: "The important detail is the preflight. The browser may receive 204 from OPTIONS but still block access if Access-Control-Allow-Origin does not exactly equal the deployed Vercel Origin. Because the app sends cookies, I cannot use wildcard origin; credentials require an explicit origin. I also made the token cookie HTTP-only, Secure, and SameSite=None in production, then ensured login and logout use matching settings. The lesson was to debug each layer - Origin, CORS response, Set-Cookie, Cookie request, then JWT middleware."

3. Deployment and frontend/backend connection

Local API URL versus deployed API routing

Status: Verified
Stage: Frontend/deployment

```
EXPECTED
Local development should target the local API while deployed client must target the hosted API/proxy.

ACTUAL
Commit 6de0843 "modified axios.ts" replaces import.meta.env.PROD check with hostname-based isLocalhost logic.

INVESTIGATION -> ROOT CAUSE -> FIX -> VERIFICATION
Current axios.ts uses VITE_API_URL/api only on localhost or 127.0.0.1; otherwise it uses /api.
frontend/vercel.json rewrites /api/:match* to https://launchlens-backend.onrender.com/api/:match*.

SOLUTION
Use a relative production /api path so Vercel rewrites it, while VITE_API_URL handles local API. Preserve
withCredentials.
```

Lesson learned: Environment-specific configuration must be explicit; a browser on Vercel cannot call localhost:5000 on the developer machine.

Deployment topology

```
Local: React http://localhost:5173 -> Axios VITE_API_URL/api -> Express http://localhost:5000
Production: Browser -> https://launch-lens-beta.vercel.app -> /api rewrite ->
https://launchlens-backend.onrender.com/api
-> Express middleware/routes/controllers/services -> Prisma -> PostgreSQL (Supabase deployment intended)
```

The exact Render build command/env var values are not committed. Verified backend scripts are **build: tsc**, **start: node dist/server.js**; server uses PORT || 5000; Prisma schema requires DATABASE_URL and DIRECT_URL. package.json includes Prisma, but no Render configuration file is present. Therefore root directory/backend build details beyond that are **Not verified**. The request states a former localhost production URL; the final Axios source proves the resolved strategy, but not the original literal value.

Vercel + Render debugging story

The Git timeline has commits "fixed render issue" and "vercel.json fixed". Current vercel.json contains two rewrites: /api/:match* to the Render backend API and a catch-all to /index.html. The second prevents client-side React routes from 404ing on refresh. A key operational caveat: the rewrite only covers /api. The tracking link uses APP_URL/r/:slug, so APP_URL must reference a host exposing the backend redirect route; confirm that runtime deployment setting rather than assuming the Vercel domain serves /r.

Frontend-backend lifecycle

```
React event -> Axios baseURL + credentials -> Vercel rewrite (production) -> Render Express
-> CORS / JSON parser / cookie parser -> route -> authenticate (where required)
-> controller -> Prisma / analytics service / AI service -> PostgreSQL or provider
-> { success, data, message } -> React context, state, cards, charts
```

4. AI integration and prompt engineering

Untrusted LLM text versus application JSON

Status: Verified
Stage: AI design

```
EXPECTED
Insight UI needs predictable objects, not arbitrary model prose.

ACTUAL
The current architecture explicitly JSON.parse()s provider text and validates it with aiInsightSchema in the
router and again in aiInsight.service.

INVESTIGATION -> ROOT CAUSE -> FIX -> VERIFICATION
An LLM can return empty, malformed, extra, or structurally invalid content. JSON mode/prompt instruction alone is
not a type guarantee.

SOLUTION
The system prompt requires JSON only; Gemini/Groq/Mistral return a string; router parses and Zod-validates before
accepting. Failed JSON is treated as a provider failure.
```

Lesson learned: Model output is untrusted input. Validate at the integration boundary and surface a controlled failure.

Provider availability and fallback

Status: Verified
Stage: AI resiliency

```
EXPECTED
One provider failure should not necessarily remove AI insight capability.

ACTUAL
AIProviderRouter loops Gemini -> Groq -> Mistral; catches errors, invalid JSON, and Zod failures.

INVESTIGATION -> ROOT CAUSE -> FIX -> VERIFICATION
AIProvider interface decouples providers. Router stops immediately after the first schema-valid result; otherwise
it accumulates errors and finally throws "All AI providers failed."

SOLUTION
Each provider implements generateInsight. Gemini uses dynamic import of @google/genai; Groq/Mistral use fetch and
json_object response format; all use temperature 0.2.
```

Lesson learned: Fallback must validate success, not merely HTTP completion. The API controller returns generic 500 if all fail.

Gemini module/timeout claims

Verified implementation: GeminiProvider dynamically imports **@google/genai** within getClient(): const { GoogleGenAI } = await import("@google/genai"). The backend package has "type": "commonjs". This is strong evidence of a module-interoperability design decision. **Not verified:** no compiler output records the claimed static-import TypeScript error, no log records buildSystemPrompt is not a function, no UND_ERR_CONNECT_TIMEOUT log/Test-NetConnection result is stored, and no source/history proves "Gemini key failed then Groq succeeded." Explain these only as unverified troubleshooting hypotheses unless you can supply terminal/network logs.

Prompt grounding evolution - verified final state

The current prompt says: use only supplied analytics; never invent clicks, visitors, conversions, revenue, causes, goals, channels, or demographics; Direct remains a classification, not evidence that users typed a URL; Unknown remains Unknown; small datasets require cautious language; do not infer a stopped promotion campaign or user intent. It limits output to a short summary, at most four insights, and at most three recommendations. Current context no longer includes countries, matching the data-quality decision.

5. Browser, country, and analytics challenges

Browser reported as Unknown

Status: Verified
Stage: Tracking data collection

```
EXPECTED
Campaign analytics should attribute recorded clicks to a browser when the header is usable.

ACTUAL
testGemini.ts contains historical sample data with browser: Unknown; current redirect controller has explicit
UAParser collection.

INVESTIGATION -> ROOT CAUSE -> FIX -> VERIFICATION
A browser field alone is not detection. The redirect must read User-Agent then parse it. Empty/unparseable headers
remain Unknown by design.

SOLUTION
redirect.controller.ts obtains req.get("user-agent") ?? "", passes it to UAParser, uses parser.getBrowser().name
when present, catches parsing failures, and writes browser into ClickEvent.
```

Lesson learned: Browser identification is header-derived and probabilistic. Brave can expose a Chrome-compatible User-Agent, so do not claim user-agent parsing proves the actual browser.

Browser interview answer

Concise: "On each public redirect I read the request User-Agent, parse it with UAParser, store the parsed browser name on ClickEvent, and aggregate it later. If the header is absent or not identifiable, I keep Unknown."

Detailed: "Browser data is useful for compatibility and channel/device behavior analysis, but it is not a fingerprint or an identity. It comes from a client-supplied header, can be reduced by privacy tools, and Brave may look like Chrome. I therefore retain Unknown instead of making a false claim."

Country remains Unknown

Status: Verified
Stage: Analytics data quality

```
EXPECTED
Country chart/AI should only present data the tracker can substantiate.

ACTUAL
schema has country String?, analytics maps null to Unknown; no IP geolocation/enrichment service or provider
appears in source. Current AI context/prompt omit country even though service still returns countries.

INVESTIGATION -> ROOT CAUSE -> FIX -> VERIFICATION
Browser detection uses an HTTP User-Agent header. Country needs a separate IP-to-geo lookup/database/service, and
storing req.ip is not enrichment.

SOLUTION
Do not claim a country fix. Current UI/AI design should treat country as unavailable/Unknown; future option is a
consent/privacy-reviewed IP enrichment pipeline.
```

Lesson learned: Data field presence is not data collection. Remove or de-emphasize an untrustworthy dimension rather than inventing detail.

Analytics transformations

Raw ClickEvent field	Transformation	Frontend/API representation
One record per redirect	count({ campaignId })	summary.totalClicks
visitorId cookie UUID	findMany distinct visitorId; length	summary.uniqueVisitors
clickedAt	ISO date YYYY-MM-DD then Map count	timeline [{date, clicks}]
device/browser/referrer/country	fetch, normalize null (Unknown/Direct), Map count	breakdown arrays/charts
analytics result	buildAIContext adds percentages by total clicks	grounded provider prompt

Repeated clicks are intentionally distinct events; a persistent same-browser cookie retains one unique visitor. Small samples make patterns preliminary. Analytics are calculated deterministically in services, rather than by the LLM. Current aggregation loads events and groups in JavaScript Maps: adequate for MVP, but a future high-volume system should use grouped SQL/rollups/caches.

6. Debugging mindset and STAR stories

Production debugging method demonstrated by LaunchLens

• Reproduce from the actual deployed origin, not only localhost.

• Read the exact response/error and identify the layer: browser policy, cookie, Express route, Prisma, provider, or UI.

• Inspect network request/response headers for CORS/cookies; inspect Git diff and current code for configuration changes.

• Compare expected and actual values, form one hypothesis, and test the smallest safe change.

• Run type/build checks, redeploy the affected service, retest from production, and verify the observable result.

Real examples: CORS/frontend-origin commits show configuration iteration; the production login commit changes SameSite and Secure handling; Axios host detection separates environments; provider router promotes invalid JSON into a recoverable provider failure; UAParser writes browser metadata before analytics aggregation.

STAR: CORS and production session

Situation: Deployment created cross-origin Vercel/Render traffic and Git history records CORS fixes followed by a production login fix. **Task:** make credentialed login/session restoration work in production. **Action:** align CORS with FRONTEND_URL, keep Axios credentials enabled, make production cookie Secure + SameSite=None, and clear it with matching options. **Result:** the final source contains the production-safe configuration and the commit records the production-login resolution.

STAR: AI reliability

Situation: Analytics insight output depends on external providers and is not safe as raw text. **Task:** preserve structured UI behavior despite invalid output/provider failure. **Action:** define AIProvider, order providers, parse/validate output in the router and service, ground prompt in deterministic analytics. **Result:** first valid provider stops the loop; all invalid/unavailable providers produce a controlled API failure instead of malformed UI data.

STAR: browser Unknown

Situation: historical test fixture shows Unknown browser values. **Task:** capture useful browser breakdown data without overstating precision. **Action:** collect User-Agent in public redirect, parse via UAParser, persist name; retain Unknown if missing/unparseable. **Result:** analytics can display an evidence-based browser dimension while documenting Brave/privacy limitations.

What I learned

A localhost success does not prove production success; CORS is origin policy, not server reachability; cookie attributes are part of authentication design; environment variables isolate deployment configuration; SDK calls still depend on provider/network configuration; model output must be validated; headers give metadata, not truth; deterministic services should calculate analytics while AI interprets facts. The exact claim that TCP connectivity differed from an API request is conceptually correct but **Not verified as a LaunchLens incident** without the mentioned timeout/test logs.

7. Top challenges and rapid-fire interview answers

Top 15 ranked problems - evidence conscious

Rank	Problem / status	Root cause -> solution
1	Production CORS - Verified	Origin configuration evolved -> FRONTEND_URL exact origin + credentials, retest network headers.
2	Production login cookie - Verified	Cross-site Lax cookie -> Secure + SameSite=None production settings.
3	Local/deployed API mismatch - Verified	Environment topology -> localhost VITE_API_URL vs production /api Vercel rewrite.
4	AI provider resilience - Verified	Provider/JSON failures -> interface, ordered fallback, JSON/Zod validation.
5	LLM hallucination risk - Verified	Raw generative output -> grounded constrained prompt + schema.
6	Browser Unknown - Verified	No/limited UA metadata -> req.get User-Agent + UAParser, retain Unknown.
7	Country Unknown - Verified	No IP geo enrichment -> omit from AI context; label data limitation.
8	Unique visitor semantics - Verified	Person vs browser confusion -> UUID cookie and distinct visitorId, document limitation.
9	Direct referrer interpretation - Verified	Null referrer ambiguity -> normalize to Direct, do not infer cause.
10	MVP aggregation scaling - Verified design limit	In-memory Maps/multiple reads -> future grouped SQL, rollups, caching.
11	Gemini static import TS error - Not verified	Dynamic import exists, but no exact error log.
12	Gemini connection timeout - Not verified	No UND_ERR_CONNECT_TIMEOUT/Test-NetConnection evidence stored.
13	buildSystemPrompt mismatch - Not verified	Export exists now, no historical error evidence.
14	Gemini key failed -> Groq succeeded - Not verified	Fallback is implemented; this exact observed run is not recorded.
15	Exact OPTIONS header mismatch - Not verified	CORS fix history supports issue class, not exact capture.

Rapid-fire Q&A;

Question	Interview-ready answer
What was the hardest verified bug?	Production cross-origin auth: CORS origin, credentials, and cookie SameSite/Secure must agree.
What is a preflight?	Browser OPTIONS check before certain cross-origin requests; a 204 alone is not success if CORS headers mismatch.
Why not * with credentials?	Credentialed CORS requires a concrete allowed origin, not wildcard.
How did you identify browser?	User-Agent -> UAParser -> ClickEvent.browser; Unknown is retained when unavailable.
Why can Brave appear as Chrome?	It may expose a Chrome-compatible User-Agent; header parsing is not proof of identity.
Why was country Unknown?	No IP-to-geolocation enrichment is implemented; req.ip storage alone does not produce country.
How does JWT work here?	Login signs userId for 7 days; middleware verifies cookie token and attaches req.user.
Why HTTP-only cookie?	It prevents normal client JavaScript from reading the token; it still needs CORS/CSRF hardening.
What if provider fails?	Router tries next provider only after catching error or invalid JSON/schema output.
Why multiple providers?	Availability/resilience; success means valid structured output, not merely an HTTP response.
How validate AI output?	JSON.parse and aiInsightSchema Zod validation in router and again in service.
How reduce hallucination?	Only analytics context, no invented conversions/revenue/causes, cautious small-data rules.
Why not LLM math?	Counts/grouping are deterministic database/service responsibilities.
All providers fail?	Router throws combined failure; controller returns controlled 500 response.
Why separate environment config?	localhost and deployed origins/routes differ; client cannot reach developer localhost from Vercel.
How does Vercel reach Render?	vercel.json rewrites relative /api to the Render API URL.
How would you scale tracking?	Fast ingest queue/batches, grouped rollups, caching, rate limits, bots and monitoring.
How add rate limiting?	Redis-backed limits keyed by IP/link/user with endpoint-specific policies.
How detect bots?	User-Agent/behavior heuristics, IP reputation, rate patterns, optional challenge; avoid treating it as perfect.

Question	Interview-ready answer
Biggest limitation?	Browser-cookie identity and simple in-memory MVP aggregation; no conversion/geo/bot pipeline.

Final one-page cheat sheet

Architecture

```
React/Vite + Axios (credentials) -> Vercel /api rewrite -> Render Express
-> CORS/JSON/cookie middleware -> JWT guard -> controller/service -> Prisma/PostgreSQL
Public /r/:slug -> visitor UUID + UAParser -> ClickEvent -> 302 destination
Analytics -> grounded context -> Gemini -> Groq -> Mistral -> JSON/Zod -> UI
```

Top 10 verified problems -> solutions

• CORS configuration iteration -> FRONTEND_URL exact origin + credentials.

• Production login -> Secure + SameSite=None cookie; matching clearCookie options.

• Local vs production API -> hostname-aware Axios and Vercel rewrite.

• Provider outage/invalid text -> AIProvider ordered fallback + parse/Zod.

• Hallucinations -> facts-only prompt and low temperature.

• Unknown browser -> parse User-Agent via UAParser; keep uncertainty.

• Unknown country -> do not claim geo; remove from AI context.

• Unique visitor ambiguity -> browser UUID, distinct count, documented limitation.

• Direct ambiguity -> classify, do not infer user action.

• MVP aggregation -> future grouped DB/rollup/cache approach.

Key commands / checks

```
Browser DevTools: Network -> OPTIONS / login -> Origin, Access-Control-Allow-Origin, Set-Cookie, Cookie
Backend: npm run build (tsc); npm start; inspect Render deployment logs
Frontend: npm run build; inspect Vercel rewrite / deployed request URL
Database: verify Prisma migration/schema and ClickEvent rows
AI: inspect provider error, raw response only in secure logs; validate JSON schema
```

Best five stories

1) Cross-origin CORS + cookie session. 2) Environment-aware Axios/Vercel proxy. 3) Validated multi-provider AI fallback. 4) User-Agent browser instrumentation with graceful Unknown. 5) Country/analytics data-quality restraint. Lead with verified evidence; label the Gemini timeout/module-error anecdotes as Not verified unless logs are available.

10 rapid answers

CORS is browser origin policy; OPTIONS 204 can still fail on headers; cookie auth needs withCredentials + exact origin + Secure/SameSite in production; User-Agent is metadata not identity; country needs separate geo enrichment; unique visitor is browser cookie not person; models do not calculate metrics; Zod validates model JSON; fallback stops at first valid provider; current system needs rate limits/bot detection before high-scale production.